

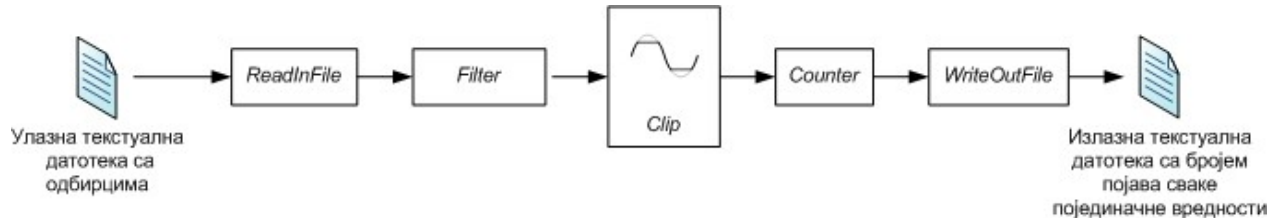
УВОЂЕЊЕ ПАРАЛЕЛИЗМА КОРИШЋЕЊЕМ OPENMP БИБЛИОТЕКЕ

Циљ вежбе је да се секвенцијална имплементација обраде сигнала оптимизује коришћењем *OpenMP* API. Потребно је идентификовати делове кода који су погодни за паралелизацију, а затим применом *OpenMP* директива постићи оптималније решење. На крају, потребно је упоредити резултате секвенцијалне и паралелне имплементације, упоредити перформансе и анализирати добијено убрзање.

Блокови обраде сигнала

Ову вежбу прате два пројекта: *Serial* и *Parallel*. У пројекту *Serial* реализована је секвенцијална имплементација блокова обраде сигнала са Слике 1. Обрада се састоји од:

- читања улазних података (*ReadInFile*),
- филтрирања одбирака из улазне датотеке филтром (*Filter*),
- ограничавања вредности на задати опсег (*Clip*),
- пребројавања вредности које су након одсецања вредности остале непромењене и вредности које су одсечене (*Counter*),
- уписа резултата у излазну датотеку (*WriteOutFile*).



Слика 1: Пример блокова обраде сигнала

Детаљи о појединачним блоковима:

- ***ReadInFile*** – Читање података (одбирака) из улазне датотеке и смештање у вектор.
- ***Filter*** – Филтрирање прочитаних одбирака изводи се тако што се из вектора у који су учитани подаци из датотеке узимају 3 суседна елемента, затим се у том прозору одређују минимална и максимална вредност, а потом се рачуна њихова разлика:

$$\text{range} = \text{max} - \text{min}$$

Резултат се уписује на одговарајуће место у **другом** вектору и представља локални распон вредности унутар три суседна узорка.

Пошто први (нулти) елемент нема елемент пре себе, као што ни последњи елемент нема елемент после себе, након примене филтра добија се вектор који има 2

елемента мање од почетног. Другим речима, резултат за прва 3 елемента улазног вектора постаје први елемент новог вектора, резултат за 2, 3. и 4. елемент постаје други елемент новог вектора, итд.

- **Clip** – Ограничавање вредности филтрираних одбирака између задатих граница. Уколико је вредност мања од доње границе, у резултат се уписује доња граница. Уколико је вредност већа од горње границе, у резултат се уписује горња граница. У супротном, вредност се не мења.
- **Counter** – Бројање елемената:
 - колико је вредности након *Clip* операције остало непромењено,
 - колико је вредности било потребно одсећи.
- **WriteToFile** – упис вредности у излазну датотеку.

Пројекат **Parallel** садржи секвенцијалну имплементацију блокова обраде идентичну као и пројекат **Serial**. У њему је потребно реализовати паралелне имплементације описаних блокова обраде, према упутствима из задатка који су дата у даљем тексту. За проверу исправности рада паралелне верзије програма користити добијену излазну датотеку и упоредити са резултатима секвенцијалне обраде.

ЗАДАТАК:

1. Изменити модул **Filter** тако да се филтрирање обавља у паралели. Метода **pushback** неће моћи да ради у паралели уколико више нити истовремено покушава да убади елемент на крај низа. Пошто је унапред познато колико ће елемената имати нови вектор (2 елемента мање од почетног вектора), потребно је урадити следеће:
 - унапред задати величину излазног вектора (најбоље у *main.cpp* пре мерења времена, због реалног поређења времена),
 - упис резултата реализовати директно по индексу,
 - паралелизовати обраду коришћењем погодне OpenMP клаузуле.
2. Изменити модул **Clip** тако да се одсецање обавља у паралели. Исто као и у претходном кораку, метода **pushback** неће радити без додатне синхронизације. Пошто је број елемената резултата унапред познат и једнак броју елемената улазног вектора, потребно је:
 - унапред задати величину излазног вектора (најбоље у *main.cpp* пре мерења времена, због реалног поређења времена),
 - сваку обрађену вредност уписати директно на одговарајућу позицију,
 - паралелизовати петљу помоћу одговарајуће OpenMP клаузуле,
 - проверити промену убрзања променом места декларације променљивих (приватне / дељене).
3. Бројање **непромењених** и **одсечених** вредности у функцији **Counter** реализовати коришћењем **редуктора** унутар паралелне **for** петље.

НАПОМЕНЕ:

- Након сваке појединачне реализоване обраде проверити исправност програма поређењем излазних датотека.
- Измерити време извршавања посебно за сваку фазу (у ***Release*** конфигурацији) као што је то урађено и у серијској имплементацији.
- У случају да имате додатне исписе за потребе отклањања грешака у коду, те исписе уклоните у крајњем мерењу времена како не би утицало на убрзање.
- Уколико је добро реализовано решење сва 3 задатка, требало би добити убрзање у сва 3 сегмента обраде сразмерно броју језгара процесора, а излазне датотеке би требало да дају бит-идентичан резултат (***outfileParallel = outfileSerial***).